$F2 \parallel WC_{\max}$ Structural Properties and Dynamic Programming

Author One[a,*], Author Two[b]

[a]Institution One, Location, Country

[b]Institution Two, Location, Country

Abstract

Minimizing $WC_{\max}$ on a two-machine flow shop, denoted $F2 \parallel WC_{\max}$, is a classical NP-hard scheduling problem. Existing exact methods rely on enumerative techniques that scale poorly with instance size, while approximation algorithms often impose restrictive assumptions on processing times. We first order all jobs globally by Johnson's rule, obtaining an initial schedule π. Scanning π from left to right, we partition it into at most K blocks: whenever a weight not seen before appears, it serves as the final job of the current block, and the next job starts a new block. We establish structural properties of optimal schedules: the permutation property, the positional constraint that block $\mathcal{G}_k$ jobs cannot appear beyond the first k blocks, and the block-end condition requiring the final job of each block to carry that block's weight (automatically satisfied by construction). The intra-block Johnson schedule inherited from π is further formalized within the DP framework. Building on these properties and the interval-based dynamic programming framework (Hall, 1994) developed for $F2|r_j|C_{\max}$, we design a polynomial-time dynamic programming algorithm with $O(|Y|_{\max} \cdot n)$ complexity that appends jobs block by block. We further show that geometric rounding of weights and processing times converts this algorithm into a polynomial-time approximation scheme.

Keywords: Flow shop scheduling; $WC_{\max}$; Dynamic programming; Structural properties

1 Introduction

2 Preliminaries

We consider the problem $F2 \parallel WC_{\max}$. Let $\mathcal{J} = \{J_1, J_2, \ldots, J_n\}$ be the set of n jobs to be processed on two machines M_1 and M_2 in series: each job J_j must be processed first on M_1 for

*Corresponding author. E-mail address: author@email.com

a_j time units and then on M_2 for b_j time units, with no preemption allowed. All processing times a_j and b_j are assumed to be positive integers. M_1 and M_2 can process at most one job at a time. Each job J_j has a weight $w_j > 0$. A schedule $\pi = (\pi(1), \pi(2), \ldots, \pi(n))$ specifies the processing sequence on both machines. For a schedule π, let $A_j = \sum_{h=1}^{j} a_{\pi(h)}$ and $B_j = \sum_{h=1}^{j} b_{\pi(h)}$ be the cumulative M_1 and M_2 processing times after the first j jobs. The completion time of the job in position j on machine M_2 is

$$C_{\pi(j)} = \max_{1 \le i \le j} \left\{ \sum_{h=1}^{i} a_{\pi(h)} + \sum_{h=i}^{j} b_{\pi(h)} \right\}. \tag{1}$$

The objective is to find a schedule π that minimizes $WC_{\max}$.

The problem $F2 \parallel WC_{\max}$ can be formulated as a mixed integer linear program (MILP). For a permutation schedule, let the positions be indexed by $k = 1, \ldots, n$. We introduce the following decision variables:

- $x_{jk} = 1$ if job J_j processing at position k, and 0 otherwise;
- C_k^1 completion time of the job at position k on M_1;
- C_k^2 completion time of the job at position k on M_2;
- C_j completion time of job J_j on M_2;
- Z the maximum weighted completion time $WC_{\max}$.
- $M = 2\sum_{j=1}^{n}(a_j + b_j)$ be a sufficiently large constant.

The MILP model is:

$$\min \quad Z \tag{2}$$

$$\text{s.t.} \quad \sum_{k=1}^{n} x_{jk} = 1, \quad j = 1, \ldots, n \tag{3}$$

$$\sum_{j=1}^{n} x_{jk} = 1, \quad k = 1, \ldots, n \tag{4}$$

$$C_1^1 = \sum_{j=1}^{n} a_j x_{j1} \tag{5}$$

$$C_k^1 = C_{k-1}^1 + \sum_{j=1}^{n} a_j x_{jk}, \quad k = 2, \ldots, n \tag{6}$$

$$C_1^2 = C_1^1 + \sum_{j=1}^{n} b_j x_{j1} \tag{7}$$

$$C_k^2 \geq C_k^1 + \sum_{j=1}^{n} b_j x_{jk}, \quad k = 2, \ldots, n \tag{8}$$

$$C_k^2 \geq C_{k-1}^2 + \sum_{j=1}^{n} b_j x_{jk}, \quad k = 2, \ldots, n \tag{9}$$

$$C_j \geq C_k^2 - M(1 - x_{jk}), \quad j = 1, \ldots, n, \; k = 1, \ldots, n \tag{10}$$

$$Z \geq w_j C_j, \quad j = 1, \ldots, n \tag{11}$$

$$x_{jk} \in \{0, 1\}, \tag{12}$$

$$C_k^1, C_k^2, C_j \geq 0 \tag{13}$$

Constraint (3)–(4) ensure a one-to-one assignment between jobs and positions. Constraints (5)–(6) compute the completion time of each position on M_1. Constraints (7)–(9) compute the completion time of each position on M_2: for $k \geq 2$, the job at position k starts on M_2 only after it completes M_1 and the preceding job releases M_2. Constraint (10) links the position-based completion times to the job-based completion times via a big-M formulation: when $x_{jk} = 1$, job J_j is at position k and $C_j \geq C_k^2$; when $x_{jk} = 0$, the right-hand side becomes $C_k^2 - M \leq 0$, which is non-binding. Constraint (11) enforces $Z \geq w_j C_j$ for every job, so that minimizing Z yields the optimal $WC_{\max}$. Constraint (12) requires the assignment variables x_{jk} to be binary. Constraint (13) imposes the nonnegativity of the completion times C_k^1, C_k^2 and C_j.

3 Problem formulation

Theorem 3.1. *$F2 \parallel WC_{\max}$ is NP-hard.*

Proof. We reduce from the Partition problem: given n positive integers $x_1, x_2, \ldots, x_r$ with $\sum_{i=1}^{r} x_i = 2T$, does there exist subset Z_1 and $Z_2 \subseteq \{1, \ldots, r\}$ such that: $\sum_{i \in Z_1} x_i = \sum_{i \in Z_2} x_i$ $= T$?

Construct an instance of $F2 \parallel WC_{\max}$, Let the number of jobs $n = r + 1$. Let $a_j = 1$, $b_j = rx_j$, $w_j = T + 1$ for jobs $j = 1, \ldots, n - 1$. Put $a_n = rT$, $b_n = 1$, $w_n = 2T + 1$ for job J_n. Does there exist a threshold Y such that:

$$WC_{\max} \leq Y = r(T + 1)(2T + 1)$$

holds?

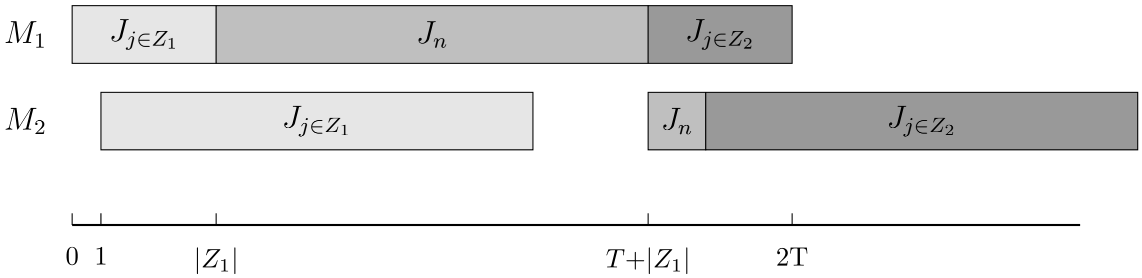

Figure 3-1

We partition jobs j_1 to j_{n-1} into two subsets via j_n, the distribution of jobs on the machine is shown in Figure 3. Since the processing times of A_1 and A_2 are unknown, while their total processing time is 2T. We may assume that the processing time of A_1 on M_2 is $r(T+\xi)$, A_2 is $r(T-\xi)$, where $-T \leq \xi \leq T$. Therefore, the completion time of J_n is determined by the following formula:

$$C_n^1 = |A_1| + rT + 1 \tag{14}$$

$$C_n^2 = 1 + r(T+\xi) + 1 \tag{15}$$

and $C_n = \max\{C_n^1, C_n^2\}$

If $\xi = 0$, then $\sum_{j\epsilon A_1} b_j = \sum_{j\epsilon A_2} b_j$, which means the partition problem has a feasible solution. In this case, the completion time of job J_n is determined by (14), we have:

$$\begin{aligned} C_n &= |A_1| + rT + 1 \leqslant (rT + r) = r(T+1) \\ w_n C_n &= (2T+1)(r(T+1)) = Y \\ WC_{\max} &= r(T+1)(C_n + rT) = Y \end{aligned}$$

so $w_j C_j \leqslant WC_{\max} = Y$

If the scheduling problem exists a threshold y such that $WC_{\max} \leq y$. When $\xi > 0$, $C_n$is determined by (15), we have:

$$\begin{aligned} C_n &= (1 + r(T+\xi) + 1) \geqslant (1 + r(T+1) + 1) \\ w_n C_n &\geqslant (2T+1)(1 + r(T+1) + 1) = Y \end{aligned}$$

When $\xi < 0$, $C_n$is determined by (14), we have:

$$\begin{aligned} C_{\max} &= (C_n + \sum_{j\epsilon A_2} b_j) \\ &= (1 + rT + |A_1| + r(T-\xi)) \\ &\geqslant (rT + r + rT) \\ &= r(2T+1) = Y \end{aligned}$$

$$WC_{\max} \geqslant r(T+1)(r(2T+1)) = Y$$

so $WC_{\max} > Y$, there exists a contradiction.

Therefore, the Partition instance admits a solution if and only if the constructed $F2 \parallel WC_{\max}$ instance admits a schedule with $WC_{\max} \leq Y$. □

Theorem 3.2. *$F2 \parallel WC_{\max}$ is strongly NP-hard.*

Proof. We reduce from the strongly NP-complete 3-Partition problem: given $3m$ positive integers $x_1, \ldots, x_{3m}$ and an integer B satisfying $B/4 < x_i < B/2$ and $\sum_{i=1}^{3m} x_i = mB$, can the integers be partitioned into m triples, each of sum B?

Let $R = 3m + 5$, $D = RB + 4$, and $T = mD$. We construct an instance with $3m$ element jobs $J_1, \ldots, J_{3m}$ and m separator jobs $S_1, \ldots, S_m$:

- for every element job J_j, set $a_j = 1$, $b_j = Rx_j + 1$;
- for every separator job S_k, set $a_{s_k} = RB + 1$, $b_{s_k} = 1$.

All processing times are positive integers. For $k = 1, \ldots, m$, define $d_k = kD + 1$ and let $Q = (T+2)^3$. Set

$$w_{s_k} = \left\lfloor \frac{Q}{d_k} \right\rfloor, \qquad w_j = \left\lfloor \frac{Q}{T} \right\rfloor \quad (j = 1, \ldots, 3m),$$

and use the decision threshold $Y = Q$. Since $d_k \leq T + 1$ and $Q \geq d_k(d_k + 1)$, we have $\lfloor Q/d_k \rfloor > d_k$. Thus, for every $d \in \{d_1, \ldots, d_m, T\}$ and every integer completion time C,

$$\left\lfloor \frac{Q}{d} \right\rfloor C \leq Q \quad \Longrightarrow \quad C \leq d. \tag{16}$$

Indeed, if $C \geq d + 1$, write $Q = qd + r$ with $q = \lfloor Q/d \rfloor$ and $0 \leq r < d$. Then $q(d+1) = Q - r + q > Q$ because $q > d > r$. Suppose first that the 3-Partition instance is a yes-instance. Let $G_1, \ldots, G_m$ be triples with $\sum_{J_j \in G_k} x_j = B$ for every k, and schedule

$$G_1, S_1, G_2, S_2, \ldots, G_m, S_m.$$

The three element jobs in G_k use 3 units on M_1, and S_k uses $RB + 1$ units. Hence S_k completes on M_1 at time kD. The three element jobs in G_k use $RB + 3$ units on M_2; since the preceding separator completes on M_2 at time $(k-1)D + 1$, the last element of G_k completes at time kD. Therefore

$$C_{s_k} = kD + 1 = d_k.$$

Every element job completes on M_2 no later than $T = mD$. By the definition of the weights, all weighted completion times are at most Q, so $WC_{\max} \leq Y$.

Conversely, suppose that a schedule π satisfies $WC_{\max} \le Q$. By the standard permutation-schedule property for two-machine flow shops with a regular objective, it suffices to consider permutation schedules. The separator jobs have identical processing times. Therefore, if their positions are fixed, exchanging the labels of two separators that occur in the wrong order can only decrease the maximum weighted completion time, because $w_{s_1} \ge w_{s_2} \ge \cdots \ge w_{s_m}$. We may consequently assume that the separators occur in the sequence $S_1, S_2, \ldots, S_m$. By (16),

$$C_{s_k} \le kD + 1 \quad (k = 1, \ldots, m), \qquad C_j \le T \quad (j = 1, \ldots, 3m).$$

No element job can occur after S_m. Indeed, if at least one element job did so, consider the last such element job. By the permutation-schedule property invoked above, the processing sequence on M_1 and M_2 coincides, so this job is also the last job on M_1; hence all m separators and all $3m$ element jobs have completed on M_1 no later than this job, and its completion on M_1 is at least the total M_1 processing time

$$m(RB + 1) + 3m = m(RB + 4) = T.$$

Its completion on M_2 would consequently be strictly greater than T, contradicting $C_j \le T$. Thus the element jobs are partitioned into the m groups G_k lying between S_{k-1} and S_k, where S_0 denotes the beginning of the schedule. Put

$$B_k = \sum_{J_j \in G_k} x_j, \qquad E_k = \sum_{h=1}^{k} |G_h|, \qquad E_0 = 0.$$

For $k = 2, \ldots, m$, the M_2 path gives

$$C_{s_k} \ge C_{s_{k-1}} + RB_k + |G_k| + 1,$$

while the M_1 path to S_{k-1} gives

$$C_{s_{k-1}} \ge (k-1)(RB + 1) + E_{k-1} + 1.$$

Combining these inequalities with $C_{s_k} \le kD + 1$ yields

$$R(B_k - B) + E_k \le 3k, \qquad k = 2, \ldots, m.$$

For $k = 1$, the M_2 path and $C_{s_1} \le D + 1$ similarly give

$$R(B_1 - B) + E_1 \le 4.$$

Because $R = 3m+5$, these inequalities imply $B_k \le B$ for every k: otherwise $B_k \ge B+1$ and the respective left-hand side is greater than $3k$ (or greater than 4 when $k = 1$). All element jobs belong to one of the groups, so $\sum_{k=1}^{m} B_k = mB$. Hence $B_k = B$ for every k. Finally, $B/4 < x_j < B/2$ implies that a group with sum B contains exactly three elements: at most two elements have sum strictly less than B, while at least four elements have sum strictly greater than B. Thus $G_1, \ldots, G_m$ is a valid 3-partition.

The construction has $4m$ jobs, and all processing times, weights, and the threshold are bounded by a polynomial in m and B. Since 3-Partition remains NP-complete when B is polynomially bounded in m, this is a strong polynomial reduction. Therefore $F2 \parallel WC_{\max}$ is strongly NP-hard. $\square$

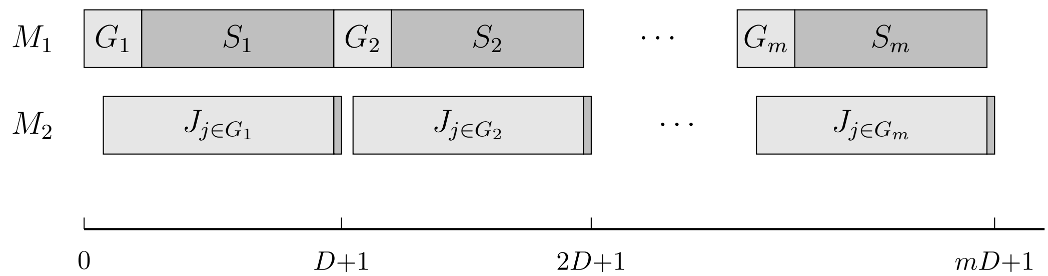

Figure 3-2: 3-Partition reduction

4 Algorithm

In this section, we design a dynamic programming algorithm for $F2 \parallel WC_{\max}$ and analyse its running time.

4.1 The number of weights is constant

For $F2 \parallel WC_{\max}$, an optimal schedule exists in which both machines process the jobs in the same sequence (Baker, 1974); we therefore restrict attention to such permutation schedules. Johnson's rule (Johnson, 1954) orders the n jobs so as to minimize the makespan: jobs with $a_j \le b_j$ are scheduled first in non-decreasing sequence of a_j, and the remaining jobs follow in non-increasing sequence of b_j.

For a given schedule π, Let $W_1 > W_2 > \cdots > W_K$ be the K distinct weights occurring among the jobs, and define $\mathcal{G}_k = \{J_j \in \mathcal{J} \mid w_j = W_k\}$, $k = 1, \ldots, K$. For $k = 1, \ldots, K$ in sequence, let ℓ_k be the position of the last job of weight W_k among the jobs not yet assigned to a block, i.e., among $\pi(\ell_{k-1}+1), \ldots, \pi(n)$, with the convention $\ell_k = \ell_{k-1}$ if no such job remains; set $\ell_0 = 0$. The blocks are then the segments

$$\mathcal{B}_k = \{\pi(\ell_{k-1}+1), \ldots, \pi(\ell_k)\}, \qquad k = 1, \ldots, K,$$

in particular, $\mathcal{B}_1 = \{\pi(1), \ldots, \pi(\ell_1)\}$. Equivalently, after the blocks $\mathcal{B}_1, \ldots, \mathcal{B}_k$ are removed, the remaining jobs are

$$\mathcal{J} \setminus \bigcup_{i=1}^{k} \mathcal{B}_i = \{\pi(\ell_k+1), \ldots, \pi(n)\}, \qquad k = 1, \ldots, K,$$

with $\ell_0 = 0$; consequently $\ell_1 \le \ell_2 \le \cdots \le \ell_K$ and no block extends past its own last job. For every block $\mathcal{B}_i$, let A_i denote the total processing time of its jobs on machine M_1, B_i their total processing time on machine M_2, C_i the completion time of its last job on machine M_2, and L_i the weight of its last job. If $\ell_k = \ell_{k-1}$, then $\mathcal{B}_k$ contains no job and is called an empty block; an empty block $\mathcal{B}_i$ has $L_i = 0$ and $A_i = B_i = C_i = 0$, and may host any job of weight at most W_i. By construction, the last job of $\mathcal{B}_k$ carries weight W_k, so the block weights are non-increasing: $W_{\mathcal{B}_1} \ge W_{\mathcal{B}_2} \ge \cdots \ge W_{\mathcal{B}_K}$.

We illustrate the block construction with a small example.

Example 4.1. *Consider a 6-job instance with sequence $\pi = (J_1, \dots, J_6)$ and the following (a_j, b_j, w_j) values:*

	J_1	J_2	J_3	J_4	J_5	J_6
a_j	2	5	6	8	4	3
b_j	5	7	9	4	3	2
w_j	2	2	5	2	4	2

The resulting groups are $\mathcal{G}_1 = \{J_3\}$, $\mathcal{G}_2 = \{J_5\}$ and $\mathcal{G}_3 = \{J_1, J_2, J_4, J_6\}$, and the blocks are $\mathcal{B}_1 = \{J_1, J_2, J_3\}$, $\mathcal{B}_2 = \{J_4, J_5\}$ and $\mathcal{B}_3 = \{J_6\}$.

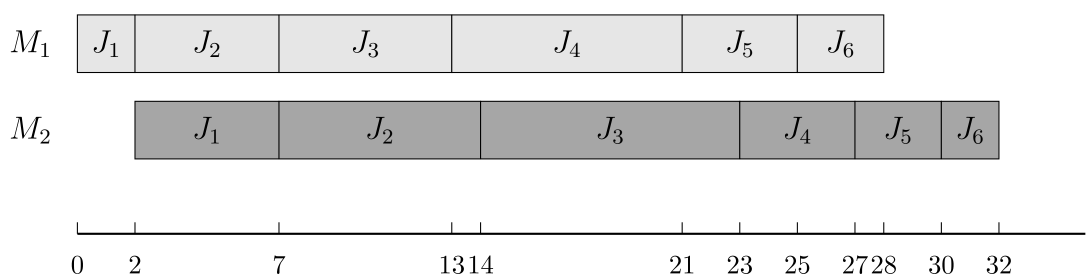

Figure 4.1

Lemma 4.1. *For any schedule π, rescheduling the jobs within each block according to Johnson's rule Johnson, 1954 and concatenating the resulting block schedules yields a schedule π' with $WC_{\max}(\pi') \le WC_{\max}(\pi)$.*

Proof. By construction, the final job of block $\mathcal{B}_k$ carries weight $W(\mathcal{B}_k)$, and every job in $\mathcal{B}_k$ has weight at most $W(\mathcal{B}_k)$: a weight occurring in $\mathcal{B}_k$ either equals $W(\mathcal{B}_k)$, or it has already appeared in an earlier block with larger weight. Let $C_k(\sigma)$ denote the completion time of the last job of $\mathcal{B}_k$ under schedule σ.

We first show that, within each block, the maximum weighted completion time is attained by the last job. For any $J_j \in \mathcal{B}_k$, the last job of the block finishes no earlier than J_j, so $C_j(\sigma) \le C_k(\sigma)$, and $w_j \le W(\mathcal{B}_k)$ by the block construction. Hence $w_j C_j(\sigma) \le W(\mathcal{B}_k) \cdot C_k(\sigma)$. Taking the maximum over $j \in \mathcal{B}_k$ yields $\max_{J_j \in \mathcal{B}_k} w_j C_j(\sigma) = W(\mathcal{B}_k) \cdot C_k(\sigma)$, and therefore

$$WC_{\max}(\sigma) = \max_{k=1,\dots,K} \{W(\mathcal{B}_k) \cdot C_k(\sigma)\}.$$

Thus the overall objective depends only on the last job of each block.

Now let π' be the schedule obtained by rescheduling the jobs within each $\mathcal{B}_k$ according to Johnson's rule, while preserving the order of the blocks. We prove $C_k(\pi') \le C_k(\pi)$ for all k by induction on k.

Base case $k = 1$. Block $\mathcal{B}_1$ starts at time zero on both machines regardless of its internal ordering, so Johnson's rule gives $C_1(\pi') \le C_1(\pi)$.

Inductive step. Assume $C_{k-1}(\pi') \le C_{k-1}(\pi)$. On M_1, the start time of $\mathcal{B}_k$ equals the total M_1 work of $\mathcal{B}_1, \ldots, \mathcal{B}_{k-1}$, which is invariant under internal rescheduling, so $\mathcal{B}_k$ begins at the same instant on M_1 under both schedules. On M_2, $\mathcal{B}_k$ starts at $\max\{\sum_{i=1}^{k} A_i,\ C_{k-1}\}$, where the first term is identical under π and π', and the second is no larger under π' by the induction hypothesis. Hence $\mathcal{B}_k$ starts no later on M_2 under π' than under π. Together with the same M_1 start and the internally optimal Johnson scheduling, this gives $C_k(\pi') \le C_k(\pi)$, completing the induction.

Finally,

$$WC_{\max}(\pi') = \max_k\ W(\mathcal{B}_k) \cdot C_k(\pi') \le \max_k\ W(\mathcal{B}_k) \cdot C_k(\pi) = WC_{\max}(\pi),$$

which establishes the lemma. $\square$

4.2 Algorithm Properties and Dynamic Programming

We now develop a dynamic programming algorithm that processes jobs sequentially in Johnson sequence and assigns each job to one of the K blocks $\mathcal{B}_1, \ldots, \mathcal{B}_K$.

Lemma 4.1 provides the structural basis for the dynamic program: given any assignment of jobs to the K blocks, sequencing the jobs within each block $\mathcal{B}_i$ according to Johnson's rule and concatenating the blocks in non-increasing weight sequence yields a schedule whose $WC_{\max}$ value is no larger than that of any other schedule respecting the same assignment. For each job J_j $(j = 1, \ldots, n)$, let $\mathscr{H}_j$ denote the set of states attainable after the first j jobs have been processed. A state records, for every block $\mathcal{B}_i$:

- A_i: the total processing time on machine M_1 of the jobs in block $\mathcal{B}_i$;
- B_i: the total processing time on machine M_2 of the jobs in block $\mathcal{B}_i$;
- C_i: the completion time of the last job of block $\mathcal{B}_i$ on machine M_2;
- L_i: the weight of the last job of block $\mathcal{B}_i$.

Since $W_1 > \cdots > W_K$, the blocks eligible for J_j form a prefix:

$$\mathcal{E}(J_j) = \{\, i : w_j \le W_i \,\}, \qquad i^* = \max_{i \in \{1,\ldots,K\}} \{\, i : W_i \ge w_j \,\},$$

so that $\mathcal{E}(J_j) = \{\, 1, \ldots, i^* \,\}$. A state is in fact the aggregate K-block tuple, described by $4K-2$ parameters: A_K and B_K need not be stored, because after the first j jobs have been processed, each of these jobs belongs to exactly one block, so the totals $\sum_{i=1}^{K} A_i = \sum_{h=1}^{j} a_h$ and $\sum_{i=1}^{K} B_i = \sum_{h=1}^{j} b_h$ are known, and A_K and B_K are recovered by

$$A_K = \sum_{h=1}^{j} a_h - \sum_{i=1}^{K-1} A_i, \qquad B_K = \sum_{h=1}^{j} b_h - \sum_{i=1}^{K-1} B_i.$$

Hence each state of $\mathscr{H}_j$ is represented by the tuple

$$(A_1, \ldots, A_{K-1};\ B_1, \ldots, B_{K-1};\ C_1, \ldots, C_K;\ L_1, \ldots, L_K).$$

The initial state set contains a single state with all components set to zero: $\mathscr{H}_0 = \{(0, \ldots, 0;\ 0, \ldots, 0;\ 0, \ldots, 0;\ 0, \ldots, 0)\}$. In Algorithm 1 below, T_{A_i} denotes the total M_1 processing time of the first i blocks, and T_{B_i} the completion time of block $\mathcal{B}_i$ on M_2.

Lemma 4.2. *Let $\mathscr{H} = (A_1, \ldots, A_{K-1}; B_1, \ldots, B_{K-1}; C_1, \ldots, C_K; L_1, \ldots, L_K)$ and $\mathscr{H}' = (A'_1, \ldots, A'_{K-1}; B'_1, \ldots, B'_{K-1}; C'_1, \ldots, C'_K; L'_1, \ldots, L'_K)$ be two states in $\mathscr{H}_j$ for job J_j. If $A_i = A'_i$, $B_i = B'_i$, $L_i = L'_i$ for all i, and $C_i \le C'_i$ for all i, then $\mathscr{H}'$ can be discarded without affecting the optimal solution.*

Proof. Consider any continuation that extends state $\mathscr{H}'$ to a complete schedule Π'. Replacing the prefix represented by $\mathscr{H}'$ with the prefix represented by $\mathscr{H}$ yields a schedule Π that uses the same assignment of the remaining jobs. For every subsequent block $\mathcal{B}_k$ ($k \ge 2$), the M_2 start time is $\max\{\sum_{i=1}^{k} A_i,\ C_{k-1}\}$, where C_{k-1} is no larger under $\mathscr{H}$ than under $\mathscr{H}'$. By induction, the M_2 completion time of every subsequent block is no larger under $\mathscr{H}$, and since A_i, B_i and L_i are identical, the objective value $WC_{\max}$ of Π does not exceed that of Π'. □

For $j = 1, \ldots, n$, the algorithm extends $\mathscr{H}_{j-1}$ to $\mathscr{H}_j$ as follows: for each state in $\mathscr{H}_{j-1}$ and each block $i \in \mathcal{E}(J_j)$, the algorithm places J_j at the end of $\mathcal{B}_i$. Only the i-th slice of the state is updated, while all other blocks remain unchanged:

$$C'_i = \max\{C_i + b_j,\ A_i + a_j + b_j\}, \quad A'_i = A_i + a_j, \quad B'_i = B_i + b_j. \tag{17}$$

The resulting candidate states are collected in temporary sets $\mathcal{Y}_1, \ldots, \mathcal{Y}_K$. By Lemma 4.2, merging the candidates removes dominated and duplicate states and yields $\mathscr{H}_j$.

Algorithm 1: Block concatenation and evaluation

Input: State $\mathscr{H} = \{A_i, B_i, C_i, L_i\}$.

Step 1. [Initialization] Set $T_{A_0} \leftarrow 0$, $T_{B_0} \leftarrow 0$, $WC_{\max} \leftarrow 0$.
Step 2. [Evaluation] **for** $i = 1$ **to** K **do**
  $T_{A_i} \leftarrow T_{A_{i-1}} + A_i$
  $T_{B_i} \leftarrow \max\{\, T_{A_{i-1}} + C_i,\ T_{B_{i-1}} + B_i \,\}$
  if $L_i > 0$ **then**
    $WC_{\max} \leftarrow \max\{\, WC_{\max},\ L_i \cdot T_{B_i} \,\}$
  end
end

Output: $WC_{\max}$.

Algorithm 2: DP for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j);max block count K; weights $W_1 > \cdots > W_K$.

Step 1. [Preprocessing] Set $L_i \leftarrow 0$ for $i = 1, \ldots, K$.
Step 2. [Initialization] Set $\mathscr{H}_i \leftarrow \{(0,0,0,0)_{i=1}^{K}\}$.
Step 3. [Generation] **for** $j = 1$ **to** n **do**
  $\mathcal{Y}_i \leftarrow \emptyset$ for $i = 1, \ldots, K$
  for *each* $(\{A_i, B_i, C_i, L_i\}_{i=1}^{K}) \in \mathscr{H}_{j-1}$ **do**
    for $i = 1$ **to** K **do**
      if $L_i = 0$ **then**
        $C_i' \leftarrow a_j + b_j$
      else
        if $w_j \leq L_i$ **then**
          $C_i' \leftarrow \max\{\, C_i + b_j,\ A_i + a_j + b_j \,\}$
          $A_i' \leftarrow A_i + a_j$, $B_i' \leftarrow B_i + b_j$, $L_i' \leftarrow w_j$
          Add the updated state to $\mathcal{Y}_i$
        end
      end
    end
  end
  [Merging] $\mathscr{H}_j \leftarrow \textsc{Merge}(\mathcal{Y}_1, \ldots, \mathcal{Y}_K)$
end
Step 4. [Feasibility check] Remove from $\mathscr{H}_n = \{A_i, B_i, C_i, L_i\}_{i=1}^{K}$ violating the positional constraint $L_i \in \{0, W_i\}$.
Step 5. [Calculate] Set $Z^* \leftarrow +\infty$
for *each state* $S = \{A_i, B_i, C_i, L_i\}_{i=1}^{K} \in \mathscr{H}_n$ **do**
  $WC_{\max} \leftarrow$ Algorithm $1\big((A_i, B_i, C_i, L_i)\big)$
  $Z^* \leftarrow \min\{\, Z^*,\ WC_{\max} \,\}$
end
Step 6. [Result] By standard backtracking Z^*.

Output: get the schedule σ.

To this end, each state in $\mathscr{H}_j$ stores a pointer to the state in $\mathscr{H}_{j-1}$ from which it was derived, together with the block $i \in \mathcal{E}(J_j)$ into which J_j was placed. Starting from the state in $\mathscr{H}_n$ attaining Z^*, backtracking along these pointers recovers, for every job J_j, the block $\mathcal{B}_i$ it belongs to. Sequencing the jobs of each block according to Johnson's rule (Lemma 4.1) and concatenating the blocks then yields an optimal schedule π^* with $WC_{\max}(\pi^*) = Z^*$.

Theorem 4.1. *Algorithm 2 solves $F2 \parallel WC_{\max}$ optimally in $O\big(n \cdot K \cdot |\mathscr{H}|_{\max}\big)$ time, where $|\mathscr{H}|_{\max} \leq 2^K \cdot V^{3K-3}$, $V = \sum_{j=1}^n a_j + \sum_{j=1}^n b_j$. For constant K, this is $O(n \cdot V^{3K-3})$.*

Proof. Each state in $\mathscr{H}_j$ is a $(4K-2)$-tuple

$$(A_1, \ldots, A_{K-1};\ B_1, \ldots, B_{K-1};\ C_1, \ldots, C_K;\ L_1, \ldots, L_K).$$

We bound the number of distinct values each component can attain.

The quantity A_i is the total M_1 processing time of jobs assigned to block $\mathcal{B}_i$, hence $A_i \in [0, V]$. Since the j jobs under consideration form a prefix of the fixed Johnson sequence, $\sum_{i=1}^{K-1} A_i$ equals the sum of the a_h values over that prefix. Consequently at most $K-1$ of the A_i are independent, contributing at most V^{K-1} distinct A-vectors. Symmetrically, $\sum_{i=1}^{K-1} B_i$ is the prefix sum of the b_j's, so the B_i components likewise contribute at most V^{K-1} distinct combinations.

The quantity C_i is the makespan of $\mathcal{B}_i$ when scheduled in isolation by Johnson's rule. Since $C_i \leq A_i + B_i \leq V$ (each block's total M_1 and M_2 work is bounded by V), the C-vector contributes at most V^{K-1} distinct values. The weight L_i equals W_i if block $\mathcal{B}_i$ is nonempty and 0 if it is empty, yielding at most 2^K distinct L-vectors.

Multiplying these bounds yields

$$|\mathscr{H}_j| \leq 2^K \cdot V^{K-1} \cdot V^{2K-2} = 2^K \cdot V^{3K-3}.$$

In iteration j, the algorithm takes each state in $\mathscr{H}_{j-1}$, tries every block $i \in \{1, \ldots, K\}$, computes the updated tuple in $O(1)$ time, and inserts it into $\mathcal{Y}_i$. The merge step collects the K sets $\mathcal{Y}_1, \ldots, \mathcal{Y}_K$ and removes duplicates in time proportional to the total number of generated tuples. Hence iteration j costs $O(K \cdot |\mathscr{H}_{j-1}|)$, and summing over all n iterations gives $O(n \cdot K \cdot |\mathscr{H}|_{\max})$.

The final concatenation via Algorithm 1 evaluates each state in $\mathscr{H}_n$ in $O(K)$ time, which is dominated by the DP phase. When K is fixed, 2^K is constant, giving $O(n \cdot V^{3K-3})$. □

4.3 A 2-Approximation Algorithm

The exact dynamic program of Section 4 runs in pseudo-polynomial time, and the FPTAS of Section 4.4 has running time exponential in the number K of distinct weights. When only a constant-factor guarantee is needed, the following elementary rule suffices.

Algorithm 3: 2-Approximation for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j).

Step 1. Sort the jobs by non-increasing weight w_j (ties broken arbitrarily).

Step 2. Schedule the jobs in this sequence on both machines.

Output: The resulting permutation schedule σ.

Theorem 4.2. *Algorithm 3 runs in $O(n \log n)$ time and returns a schedule with $WC_{\max} \leq 2\,WC^*_{\max}$.*

Proof. Let σ be the schedule returned by Algorithm 3, let j_k be the job in position k of σ, and let $S_k = \{j_1, \ldots, j_k\}$. Since the jobs are sorted by non-increasing weight, every job of S_k has weight at least w_{j_k}.

We first bound the completion times in σ. By (1), the job in position k completes on M_2 at time $C_{j_k}(\sigma)$, and each term in the maximum is bounded by the full sum:

$$\sum_{h=1}^{i} a_{j_h} + \sum_{h=i}^{k} b_{j_h} \leq \sum_{h=1}^{k} a_{j_h} + \sum_{h=1}^{k} b_{j_h} = \sum_{h \in S_k} \big(a_h + b_h\big), \qquad 1 \leq i \leq k.$$

Hence

$$C_{j_k}(\sigma) \;\leq\; \sum_{h \in S_k} \big(a_h + b_h\big).$$

Since $WC_{\max}(\sigma) = \max_{1 \leq k \leq n} w_{j_k}\, C_{j_k}(\sigma)$, multiplying the inequality $C_{j_k}(\sigma) \leq \sum_{h \in S_k}(a_h + b_h)$ by $w_{j_k} > 0$ and taking the maximum over k gives

$$WC_{\max}(\sigma) = \max_{1 \leq k \leq n} w_{j_k}\, C_{j_k}(\sigma) \leq \max_{1 \leq k \leq n} w_{j_k} \sum_{h \in S_k} \big(a_h + b_h\big).$$

It remains to bound the right-hand side by $2WC^*_{\max}$. Fix a prefix S_k and let π^* be an optimal schedule. Let u be the job of S_k that finishes last on M_1 in π^*, and v the job of S_k that finishes last on M_2. All jobs of S_k are processed on M_1 before u completes M_1, so $C^*_u \geq \sum_{h \in S_k} a_h$; likewise, all M_2 processing of the jobs in S_k precedes v's completion on M_2, so $C^*_v \geq \sum_{h \in S_k} b_h$. Since $u, v \in S_k$, we have $w_u, w_v \geq w_{j_k}$. Because $WC^*_{\max}$ bounds the weighted completion time of every job in π^*, the bound $C^*_u \geq \sum_{h \in S_k} a_h$ together with $w_u \geq w_{j_k}$ gives

$$WC^*_{\max} \geq w_u\, C^*_u \geq w_{j_k} \sum_{h \in S_k} a_h,$$

and, similarly, $C^*_v \geq \sum_{h \in S_k} b_h$ together with $w_v \geq w_{j_k}$ gives

$$WC^*_{\max} \geq w_v\, C^*_v \geq w_{j_k} \sum_{h \in S_k} b_h.$$

Combining these two lower bounds yields

$$WC^*_{\max} \geq w_{j_k} \max\Big\{ \sum_{h \in S_k} a_h,\ \sum_{h \in S_k} b_h \Big\} \geq \frac{w_{j_k}}{2} \Big(\sum_{h \in S_k} a_h + \sum_{h \in S_k} b_h \Big),$$

where the last inequality uses $\max\{x, y\} \geq (x + y)/2$. Hence $w_{j_k} \sum_{h \in S_k}(a_h + b_h) \leq 2WC^*_{\max}$ for every k, and the claim follows from the upper bound above. $\square$

4.4 FPTAS

We now convert the exact dynamic program of Algorithm 2 into a fully polynomial-time approximation scheme by rounding the input data (Hall, 1994). One key point of this technique is the determination of valid lower and upper bounds for the coordinates to be rounded: each time coordinate ranges in a bounded interval, and rounding it to one of a fixed number of geometrically spaced values keeps the relative error within a single factor δ. The total processing times A_i, B_i of block $\mathcal{B}_i$ on M_1 and M_2, and its completion time C_i, are rounded. The weight L_i of the last job of block $\mathcal{B}_i$ is not rounded; following Algorithm 2, the positional constraint $L_i \in \{0, W_i\}$ is required for every block, and a state violating it is discarded.

Fix $0 < \varepsilon < 1$ and let

$$\delta = 1 + \frac{\varepsilon}{2n}, \qquad V_A = \sum_{j=1}^{n} a_j, \qquad V_B = \sum_{j=1}^{n} b_j, \qquad V = V_A + V_B.$$

For every block $\mathcal{B}_i$, the recurrence (17) preserves $A_i \leq V_A$ and $B_i \leq V_B$, since block $\mathcal{B}_i$ contains a subset of the jobs; moreover $C_i \leq A_i + B_i \leq V$. We therefore round the A dimension over $[0, V_A]$, the B dimension over $[0, V_B]$, and the C dimension over $[0, V]$. To this end, for $x \in \{A, B, C\}$ set $v^x = \lceil \log_\delta V_x \rceil$, where $V_C = V$, and split the range $[0, V_x]$ of dimension x into the $v^x + 1$ intervals

$$I^x_0 = [0, 1), \qquad I^x_r = [\delta^{r-1}, \delta^r) \;\; (1 \leq r \leq v^x - 1), \qquad I^x_{v^x} = [\delta^{v^x - 1}, V_x].$$

All processing times are integral, so every value taken by a coordinate A_i, B_i, C_i is a non-negative integer. Hence any two values lying in a common interval satisfy $x \leq \delta\, y$; we call this the *interval-dominance* property, and it is the only fact about the rounding used in the analysis below.

For each block $\mathcal{B}_i$, let $r^C_i \in \{0, \ldots, v^C\}$, $r^A_i \in \{0, \ldots, v^A\}$ and $r^B_i \in \{0, \ldots, v^B\}$ be the indices of the intervals containing C_i, A_i and B_i, respectively. The *identifier* of a state $s = (A_i, B_i, C_i, L_i)_{i=1}^K$ is the configuration of interval indices

$$\ell(s) = (\mathbf{r}^A; \mathbf{r}^B; \mathbf{r}^C),$$

where $\mathbf{r}^x = (r^x_1, \ldots, r^x_K) \in \{0, \ldots, v^x\}^K$ for $x \in \{A, B, C\}$. There are $(v^A + 1)^K (v^B + 1)^K (v^C + 1)^K$ labels, a number independent of the job sizes; since $v^A, v^B \leq v^C$, this is at most $(v^C + 1)^{3K}$. The scheme keeps at most one state of each label: when several states share a label, only one of them is retained. The weight label L_i is the weight of the last job of block $\mathcal{B}_i$ and is not rounded.

Algorithm 4: State-class FPTAS for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j); block count K; accuracy ε.

Step 1. [Preprocessing] Set $\delta = 1 + \varepsilon/(2n)$, $V_A = \sum_j a_j$, $V_B = \sum_j b_j$, $V = V_A + V_B$, $v^x = \lceil \log_\delta V_x \rceil$ for $x \in \{A, B, C\}$ with $V_C = V$, and the interval families I_r^x.

Step 2. [Initialization] Set $\mathscr{S}_0 \leftarrow \{s^0\}$, where s^0 is the zero state.

$(A_i, B_i, C_i, L_i)_{i=1}^K = (0,0,0,0)_{i=1}^K$; set $L_i \leftarrow 0$ for $i = 1, \dots, K$.

Step 3. [Generation] **for** $j = 1$ **to** n **do**

- $\mathcal{Y}_i \leftarrow \emptyset$ for $i = 1, \dots, K$
- **for** *each state* $s = (A_i, B_i, C_i, L_i)_{i=1}^K \in \mathscr{S}_{j-1}$ **do**
  - **for** $i = 1$ **to** K **do**
    - **if** $L_i = 0$ **then**
      - $\tilde{C}_i = a_j + b_j$
    - **else**
      - **if** $w_j \le L_i$ **then**
        - $\tilde{C}_i = \max\{\, C_i + b_j,\ A_i + a_j + b_j \,\}$
        - $\tilde{A}_i = A_i + a_j$, $\tilde{B}_i = B_i + b_j$, $\tilde{L}_i = w_j$
        - Add the state $\tilde{s} = (\tilde{A}_i, \tilde{B}_i, \tilde{C}_i, \tilde{L}_i)_{i=1}^K$ to $\mathcal{Y}_i$
      - **end**
    - **end**
  - **end**
- **end**
- [Elimination] Among the states of $\mathcal{Y}_1, \dots, \mathcal{Y}_K$ sharing the same label $\ell(s) = (\mathbf{r}^A; \mathbf{r}^B; \mathbf{r}^C)$, keep only one; set $\mathscr{S}_j$ to the kept states.

end

Step 4. [Result] Discard every state in $\mathscr{S}_n$ that violates the positional constraint ($L_i \notin \{0, W_i\}$ for some block), and evaluate the remaining ones with Algorithm 1; return the schedule attaining the minimum value.

Output: An approximate schedule σ.

Lemma 4.3. *Let $s(j^*) = (A(j^*)_i, B(j^*)_i, C(j^*)_i, L(j^*)_i)_{i=1}^K$, $j = 0, \dots, n$, be the states along an optimal trajectory of Algorithm 2 with final value $WC^*_{\max}$. After processing j jobs, the state set $\mathscr{S}_j$ of Algorithm 4 contains a state $\hat{s}_j = (\hat{A}_i, \hat{B}_i, \hat{C}_i, \hat{L}_i)_{i=1}^K$ satisfying*

$$\hat{A}_i \le A(j^*)_i\, \delta^j, \qquad \hat{B}_i \le B(j^*)_i\, \delta^j, \qquad \hat{C}_i \le C(j^*)_i\, \delta^j \qquad (1 \le i \le K). \tag{18}$$

Proof. We proceed by induction on j. The base case $j = 0$ is the zero state s^0 of Step 2 of Algorithm 4. Assume (18) holds for j, and let i^+ be the block to which $s(j^*)$ appends J_{j+1} in Algorithm 2. By the positional constraint of Algorithm 2, which assigns a job only to a block i with $w \le W_i$, we have $w_{j+1} \le W_{i^+}$; this constraint depends only on J_{j+1} and the block index, so the same transition is applicable from $\hat{s}_j$. Applying the exact recurrences to $\hat{s}_j$ and using $\delta^j \ge 1$ gives a candidate $\tilde{s}_{j+1}$ with

$$\tilde{A}_{i^+} = \hat{A}_{i^+} + a_{j+1} \le \big(A(j^*)_{i^+} + a_{j+1}\big)\delta^j = A((j{+}1)^*)_{i^+}\delta^j,$$

$$
\begin{aligned}
\tilde{B}_{i^+} &= \hat{B}_{i^+} + b_{j+1} \le \big(B(j^*)_{i^+} + b_{j+1}\big)\delta^j = B((j{+}1)^*)_{i^+}\delta^j, \\
\tilde{C}_{i^+} &= \max\big\{\hat{C}_{i^+} + b_{j+1},\ \hat{A}_{i^+} + a_{j+1} + b_{j+1}\big\} \\
&\le \max\big\{C(j^*)_{i^+} + b_{j+1},\ A(j^*)_{i^+} + a_{j+1} + b_{j+1}\big\}\delta^j = C((j{+}1)^*)_{i^+}\delta^j,
\end{aligned}
$$

while the coordinates of the remaining blocks are unchanged and satisfy the bound at $j+1$. The candidate $\tilde{s}_{j+1}$ carries the label $\ell(\tilde{s}_{j+1})$. By interval dominance, the state $\hat{s}_{j+1}$ kept in $\mathscr{S}_{j+1}$ with that label satisfies $\hat{x}_i \le \delta\,\tilde{x}_i$ for every $x \in \{A, B, C\}$ and every i; this yields (18) at $j+1$ and completes the induction. Since every state of $\mathscr{S}_n$ stores all $3K$ time coordinates, the induction covers the coordinates of the last block $\mathcal{B}_K$ as well. These coordinates are rounded and stored rather than recovered by prefix-sum subtraction as in the exact dynamic program, because the recovered values would accumulate the rounding errors of the first $K-1$ blocks and destroy the factor δ^j in (18). □

Theorem 4.3. *For every $0 < \varepsilon < 1$, Algorithm 4 returns a schedule with*

$$
WC_{\max} \;\le\; (1+\varepsilon)\, WC^*_{\max},
$$

where $WC^*_{\max}$ is the optimal value of $F2 \parallel WC_{\max}$.

Proof. Let $s(n^*)$ be the final state of the optimal trajectory of Algorithm 2 and $\hat{s}_n$ the state of $\mathscr{S}_n$ guaranteed by Lemma 4.3 at $j = n$. In the block-concatenation evaluation of Algorithm 1, the quantity $T_{B,i}$ is nondecreasing in every coordinate and positively homogeneous, so $T_{B,i}(\hat{s}_n) \le \delta^n\, T_{B,i}(s(n^*))$. Since the block weights W_i are fixed,

$$
WC_{\max}(\hat{s}_n) = \max_{1\le i\le K} W_i\, T_{B,i}(\hat{s}_n) \le \delta^n \max_{1\le i\le K} W_i\, T_{B,i}(s(n^*)) = \delta^n\, WC^*_{\max}.
$$

Finally, $\delta^n = (1+\varepsilon/(2n))^n \le 1+\varepsilon$ for $0<\varepsilon<1$. The schedule returned by Algorithm 4 is no worse than the schedule obtained by concatenating the blocks of the state $\hat{s}_n$, so its value is at most $(1+\varepsilon)\, WC^*_{\max}$. □

Theorem 4.4. *For fixed K, Algorithm 4 runs in*

$$
O\!\left(n^{3K+1} \cdot K \cdot \frac{(\log V)^{3K}}{\varepsilon^{3K}}\right)
$$

time and is therefore a fully polynomial-time approximation scheme for $F2 \parallel WC_{\max}$.

Proof. At each level j, Algorithm 4 keeps at most one state of each label, and there are $(v^A+1)^K(v^B+1)^K(v^C+1)^K \le (v^C+1)^{3K}$ labels. Each state generates K candidates in $O(1)$ time, and the final evaluation of the states of $\mathscr{S}_n$ costs $O(K)$ per state. Hence the running time is

$$
O\big(nK\,(v^C+1)^{3K}\big).
$$

Since $\ln(1+\varepsilon/(2n)) \ge \varepsilon/(4n)$ for $0<\varepsilon<1$,

$$
v^C = \left\lceil \frac{\ln V}{\ln \delta} \right\rceil = O\!\left(\frac{n \log V}{\varepsilon}\right),
$$

and for constant K the bound reduces to $O(n^{3K+1}K(\log V)^{3K}/\varepsilon^{3K})$, polynomial in n, $\log V$, and $1/\varepsilon$. □

Weight rounding. The FPTAS above rounds the time coordinates A_i, B_i, C_i and assumes that the number K of distinct weights is fixed. An analogous geometric rounding applies to the weights themselves when their ratio is bounded. Let $W_{\min} = \min_j w_j$, $W_{\max} = \max_j w_j$, and suppose $W_{\max}/W_{\min} \le c$ for a constant c. Partition the interval $[W_{\min}, W_{\max}]$ into

$$J_0 = [W_{\min}, W_{\min}\delta), \qquad J_r = [W_{\min}\delta^r, W_{\min}\delta^{r+1}) \;\; (1 \le r \le q-1), \qquad J_q = [W_{\min}\delta^q, W_{\max}],$$

with $q = \lceil \log_\delta(W_{\max}/W_{\min}) \rceil$, and round each weight w_j upward to the right endpoint of its interval,

$$\hat{w}_j = W_{\min}\,\delta^{r_j+1}, \qquad r_j = \left\lfloor \log_\delta \frac{w_j}{W_{\min}} \right\rfloor,$$

so that $w_j \le \hat{w}_j \le \delta\, w_j$. The rounded weights take only $q+1 = O\big(\log_\delta(W_{\max}/W_{\min})\big)$ distinct values, so jobs whose weights differ by a factor at most δ merge into a single class. Since $\hat{w}_j \le \delta w_j$ for every j, the optimal schedule $\hat{\sigma}$ of the rounded instance satisfies

$$WC_{\max}(\hat{\sigma}) \le \hat{W}C_{\max}(\hat{\sigma}) = \hat{W}C^*_{\max} \le \hat{W}C_{\max}(\pi^*) \le \delta\, WC^*_{\max},$$

where π^* is an optimal schedule of the original instance. Hence the weight rounding incurs a relative error of at most $\delta = 1 + \varepsilon/(2n) \le 1 + \varepsilon$, exactly as in the completion-time rounding.

5 Heuristic Algorithms

The exact dynamic program of Section 4 is pseudo-polynomial, and the running time of the FPTAS of Section 4.4 is exponential in the number K of distinct weights. For large instances, fast heuristics are therefore of practical interest. In this section we present two heuristics for $F2 \parallel WC_{\max}$: an adaptation of the NEH algorithm (Nawaz et al., 1983) and an ant colony optimization (ACO) algorithm (Dorigo et al., 1996).

Throughout this section, the objective of a permutation $\sigma = (\sigma(1), \ldots, \sigma(k))$ of k jobs is evaluated by (1), so that

$$WC_{\max}(\sigma) = \max_{1 \le t \le k} w_{\sigma(t)}\, C_{\sigma(t)},$$

which is computed in $O(k)$ time.

5.1 NEH

The NEH algorithm (Nawaz et al., 1983) is one of the best-performing constructive heuristics for flow-shop scheduling. It orders the jobs by decreasing priority and then inserts them one by one into the partial schedule at the position that minimizes the objective.

For $F2 \parallel WC_{\max}$, the natural priority is the weight: jobs of large weight should be scheduled early, because each job contributes to the objective through the product w_jC_j. We therefore sort the jobs by non-increasing weight, breaking ties by non-increasing total processing time $a_j + b_j$. The k-th job is inserted into each of the k possible positions of the current partial schedule, and a position attaining the smallest $WC_{\max}$ is kept.

Algorithm 5: NEH heuristic for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j).

Step 1. [Sequence] Sort the jobs by non-increasing weight w_j, breaking ties by non-increasing $a_j + b_j$; let $J_1, \ldots, J_n$ be the resulting sequence.
Step 2. [Initialization] $\sigma \leftarrow (J_1)$.
Step 3. [Insertion] **for** $k = 2$ **to** n **do**
  $\sigma \leftarrow$ the sequence obtained by inserting J_k into σ at a position minimizing $WC_{\max}$.
end

Output: Schedule σ.

Theorem 5.1. *Algorithm 5 runs in $O(n^3)$ time.*

Proof. At step k, the job J_k is inserted into each of the k positions of a partial schedule of $k-1$ jobs, and each candidate sequence is evaluated in $O(k)$ time by the recurrence above. Hence step k costs $O(k^2)$, and the total running time is $O\big(\sum_{k=1}^{n} k^2\big) = O(n^3)$. □

6 Conclusion

References

Baker, K. R. (1974). *Introduction to Sequencing and Scheduling*. Wiley, New York.

Dorigo, M., Maniezzo, V., and Colorni, A. (1996). Ant system: optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, 26(1):29–41.

Hall, L. A. (1994). A polynomial approximation scheme for a constrained flow-shop scheduling problem. *Mathematics of Operations Research*, 19(1):68–85.

Johnson, S. M. (1954). Optimal two- and three-stage production schedules with setup times included. *Naval Research Logistics Quarterly*, 1(1):61–68.

Nawaz, M., Enscore, E. E., and Ham, I. (1983). A heuristic algorithm for the m-machine, n-job flow-shop sequencing problem. *Omega*, 11(1):91–95.